

Tarea Extra-clase: Laboratorio de rendimiento de estructuras de datos

Andrés Emiliano Aguilar Méndez

Fabrizio Alejandro Arias López

Emmanuel Martín Rojas Navarro

José Pablo Valverde Durán

Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica

CE1103: Algoritmos y Estructuras de Datos I

Profesor: MSc. Andrés Vargas Rivera

9 de abril de 2026

1. ESTRUCTURAS EVALUADAS

- Array
- Lista Simplemente Enlazada
- Lista Doblemente Enlazada
- Double Ended List
- Lista Circular
- Stack (Array y Lista)
- Queue (Array y Lista)

2. METODOLOGÍA

Para hacer las pruebas, se trabajó con distintos tamaños de datos: 10, 100, 1.000 y 10.000 elementos. Esto permitió ver cómo se comportan las estructuras a pequeña y gran escala. Además, cada experimento se repitió 5 veces para obtener resultados más confiables y evitar que una sola medición afectara las conclusiones.

En cuanto a lo que se midió, se tomó en cuenta el tiempo de ejecución en nanosegundos y el uso de memoria en bytes. Así se puede comparar no solo qué tan rápido es cada proceso, sino también cuántos recursos consume.

También se evaluaron varias operaciones importantes. Entre ellas están la inserción al inicio, al final y en posiciones intermedias, para ver cómo responde cada estructura en distintos casos. Además, se analizaron la eliminación de elementos y la búsqueda, que son operaciones bastante comunes. Por último, se incluyeron el acceso

por índice, el reemplazo de datos y el uso de memoria, con el fin de tener una visión más completa del rendimiento general.

3. RESULTADOS PRINCIPALES

3.1. Comparación 1: Arrays vs Listas Enlazadas

Acceso por Índice ($O(1)$ vs $O(n)$)

Tamaño	Array	Lista	Lista Doble	Double	Lista
(ns)		Simple (ns)	(ns)	Ended (ns)	Circular (ns)
10	3.800	4.400	4.900	6.700	5.300
100	4.400	60.500	31.700	36.300	76.100
1.000	61.900	1.374.000	1.112.300	837.800	1.383.400
10.000	359.300	57.536.700	32.870.200	30.241.600	59.675.000

Hallazgo: Array es exponencialmente más rápido. Lista Doble y Double Ended muestran mejor desempeño que Lista Simple debido a búsqueda bidireccional.

Búsqueda ($O(n)$ en todas)

Tamaño	Array (ns)	Lista	Lista Doble	Double	Lista
		Simple (ns)	(ns)	Ended (ns)	Circular (ns)
10	12.800	49.000	13.500	18.700	22.500
100	205.100	216.700	268.000	166.300	220.800
1.000	2.998.000	2.537.100	2.537.500	2.514.100	3.223.600
10.000	22.054.300	67.938.700	68.781.300	67.882.900	69.997.300

Hallazgo: Complejidad lineal confirmada. Array mantiene ventaja en cache locality. Listas enlazadas más lentas en tamaños grandes por saltos de memoria.

Inserción al Inicio ($O(1)$ vs $O(n)$)

Tamaño	Array (ns)	Lista Simple (ns)	Lista Doble (ns)	Double Ended (ns)	Lista Circular (ns)
10	6.300	10.300	5.200	6.300	31.700
100	147.300	11.000	12.600	12.200	19.200
1.000	2.554.600	99.100	131.900	137.100	177.300
10.000	40.138.700	453.700	587.400	654.100	422.300

Hallazgo: Array sufre masivamente con inserciones al inicio (desplazamiento de n elementos). Listas enlazadas $O(1)$ permanecen eficientes. Lista Circular inicial lenta.

Inserción al Final ($O(n)$ vs $O(1)$)

Tamaño (ns)	Array (ns)	Lista Simple (ns)	Lista Doble (ns)	Double Ended (ns)	Lista Circular (ns)
10	9.500	5.800	5.000	4.300	6.400
100	10.500	16.800	44.300	25.200	27.100
1.000	64.200	1.482.700	62.000	48.800	47.400
10.000	460.000	57.609.900	674.300	528.900	468.100

Hallazgo: Array muy eficiente. Lista Simple problema critical: $O(n)$ al recorrer hasta el final. Double Ended y Circular optimizados para inserción final.

Eliminación ($O(n)$)

Tamaño	Array (ns)	Lista Simple (ns)	Lista Doble (ns)	Double Ended (ns)	Lista Circular (ns)
10	9.100	9.100	9.700	10.300	7.300
100	185.700	22.300	18.800	17.400	16.700
1.000	2.790.800	824.200	370.600	134.900	197.900
10.000	38.439.200	58.319.500	769.000	645.400	1.111.600

Hallazgo: Array requiere desplazamiento ($O(n)$). Lista Doble y Double Ended más eficientes. Array catastrófico en tamaños grandes.

Uso de Memoria (bytes)

Estructura	10 elementos	100 elementos	1.000 elementos	10.000 elementos
Array	56	416	4.016	40.016
Simple Lista	240	2.400	24.000	240.000
Doble Lista	280	2.800	28.000	280.000
Double Ended	280	2.800	28.000	280.000
Circular Lista	240	2.400	24.000	240.000

Hallazgo: Array es 10x más eficiente en memoria. Listas enlazadas tienen overhead de punteros (~24 bytes por nodo).

3.2. Comparación 2: Stack vs Queue en Escenarios Reales

Escenario 1: Undo (LIFO - Stack Correcto)

Tamaño	Stack Array (ns)	Stack Lista (ns)	Queue Array (ns)	Queue Lista (ns)
10	10.100	11.600	12.800	9.500
100	39.800	55.300	40.200	18.000
1.000	238.000	195.500	197.900	45.200
10.000	1.171.300	1.415.200	1.147.700	1.016.700

Análisis:

Stack + Array: Rendimiento consistente, óptimo para historial

Queue (incorrecto): Comportamiento igual pero no refleja semántica Undo

Conclusión: Stack Array es 1.15x más rápido que Stack Lista a 10.000 elementos

Escenario 2: Fila de Atención (FIFO - Queue Correcto)

Tamaño	Queue Array (ns)	Queue Lista (ns)	Stack Array (ns)	Stack Lista (ns)
10	12.800	6.800	3.400	3.000
100	40.100	15.200	8.800	11.400
1.000	196.900	45.200	83.000	61.800
10.000	1.130.700	1.275.800	506.100	584.500

Análisis:

Queue Array: 1.13x más rápido inicialmente, pero requiere reallocation

Queue Lista: Escalabilidad $O(1)$ garantizada, mejor a largo plazo

Stack (incorrecto): Push/pop rápido, pero servicio LIFO es inadecuado

4. ANÁLISIS E INTERPRETACIÓN

4.1. Comportamiento

Los arrays ofrecen acceso en $O(1)$, ideal para acceso aleatorio o búsqueda en datos ordenados, pero las inserciones/eliminaciones al inicio cuestan $O(n)$ por el desplazamiento de elementos.

Las listas simplemente enlazadas invierten ese balance: acceso secuencial en $O(n)$, pero inserción al inicio en $O(1)$, lo que las hace flexibles para estructuras dinámicas.

Las listas doblemente enlazadas agregan recorrido bidireccional e inserciones en $O(1)$, a costa de mayor consumo de memoria por el doble puntero por nodo.

Los deque permiten inserciones y eliminaciones en ambos extremos en $O(1)$, aunque el acceso a elementos intermedios sigue siendo $O(n)$.

Las listas circulares optimizan la inserción al final en $O(1)$ y son útiles para rotaciones o colas circulares, pero el acceso general sigue en $O(n)$.

Trade-offs clave: los arrays pueden ser hasta 100× más rápidos en acceso, pero hasta 40× más lentos en inserciones. Las listas enlazadas consumen hasta 10× más memoria por los punteros, pero garantizan inserciones en tiempo constante.

Para stacks y queues, ambos operan en $O(1)$, aunque implementados con arrays las colas pueden sufrir realocaciones de memoria; con listas enlazadas el comportamiento es más predecible.

Caso de Uso	Estructura Recomendada	Justificación
Acceso frecuente por índice	Array	$O(1)$, cache-friendly
Inserción/Eliminación frecuente	Lista Doble	$O(1)$ ops, búsqueda bilateral
Undo/Redo	Stack+Array	$O(1)$, bajo overhead
Fila de atención	Queue+Lista	Escalabilidad $O(1)$

Caso de Uso	Estructura Recomendada	Justificación
Deque (ambos extremos)	Double Ended	Optimizado para casos bipolares
Datos ordenados	Array	Mejor para búsqueda binaria
Tamaño desconocido	Lista Simple	Dinámico, sin pre-allocación

5. CONCLUSIONES

No existe estructura universal, cada estructura es óptima para escenarios específicos. Complejidad teórica vs práctica: Arrays superan listas en tamaños pequeños por cache locality, a pesar de complejidad $O(n)$ vs $O(1)$. Overhead de memoria: Listas enlazadas tienen costo 10x mayor pero justificado por flexibilidad. Stack vs Queue: Implementación con listas supera arrays para Queue FIFO por ausencia de reallocations. Escalabilidad: Operaciones $O(n)$ se vuelven críticas a 10.000 elementos (diferencia de 1000x). Diseño correcto importa: Usar Queue para FIFO es 2-3x más eficiente que Stack LIFO en aplicaciones reales.